ALPEDIA ／ PHASE 2 INTERIM REPORT

Alpedia

フェーズ2 中間報告
ここまでの達成と、これからのアクション

対象期間：フェーズ1 完了後〜2026年10月15日

対象：Windows 版（Python + Ursina/Panda3D、物理の核は Rust）

前回：フェーズ1 開発完了報告（2026年10月）



要約

この期間のまとめ — 核の分離と Rust 化が前倒しで進み、見た目も一段くっきりした

フェーズ2 前半の「試作と選定」のうち、柱1（核の分離・軽量化）と柱3（見た目）は計画より先へ進んだ。
Rust の物理コアは試作にとどまらず、今のゲームに組み込んで配布できる状態になった。柱2（マルチプレイ）はまだ手を付けていない。

7.9 倍
構築物の物理計算の速さ
（チュートリアル、Python 版比）

3.8 倍
ゲーム全体の1ティック
71.9 → 18.8 ms

約 1/2
描画準備の時間
14.0 → 7.3 ms／フレーム

0 件
Python 版との動きの違い
（自動テスト 35 件・場面比較）

できたこと

- コア（物理・世界）と画面・MOD をはっきり分けた
- サバイバル・街を MOD にして、MOD 選択画面で選ぶ形に統一
- 物理の核を Rust 化（部品 → 1サブステップまるごと）
- Windows でダブルクリックでビルドできる手順

見た目・品質

- ピクセルアートのテクスチャ・輪郭の線・角の陰
- アクチュエーターを回しても形がねじれない
- 重さの内訳を画面とログに表示
- エディタが開かない・地形に板が重なる等 5 件を修正

これから

- 構築物どうしの押し出しなど、残る重い所を Rust へ
- GPU 側の重さ（描画・待ち 12〜20 ms）を減らす
- LAN ホスト型マルチプレイの最小試作に着手
- 画面下のヒント表示と、形を見分けるテスト

フェーズ2 前半の計画と、いまの進み具合

フェーズ1 報告で決めた「フェーズ2 前半（〜2027 Q1）」の項目ごとの状態。

区分	計画した項目	状態	いまの中身
核の試作と選定	物理の芯を Rust で試作し、Python 版と結果を比べる	✓達成	試作を越えてゲーム本体に組み込み済み。比較テストで動きが同じことを確認
	WASM でブラウザでも同じ結果になるか確かめる	○未着手	Rust の核は描画に依存しない形になったので、次に試せる
	決まった順・刻みの計算と再現テスト	△一部	Python 版と Rust 版の一致テストはある。同じ入力で必ず同じ結果になる保証はまだ
	描画エンジンの候補を同じ場面で比べる	○未着手	比べるための「重さ」の計測表示は用意した
マルチプレイの試作	LAN のホスト型（2〜4 人）／送る内容／測り方	○未着手	次の 4 週間で最小試作に着手する
	セーブを「ワールド」と「プレイヤー」に分ける	○未着手	セーブ形式に版番号（format 2）と自動の読み替えを入れ、分ける準備はできた
見た目と UI	面ごとの UV とくっきりしたテクスチャ集	✓達成	三角形ごとの投影と 16×16 のピクセルアート 9 枚
	角の丸みを小さく、縁の線と軽い陰	△一部	線と陰は達成。角の丸みは当たり判定と合わせるため、今回は見送り（判断は p.16）
	画面下のヒント表示、デザインの決まり	○未着手	次の 4 週間で着手
	シナリオの台本と章ごとの目的	○未着手	次の 4 週間で第1章の台本
計画外で実施	コアと MOD の分離／サバイバル・街の MOD 化／重さの表示／不具合修正	＋追加	Rust 化と、今後のマルチプレイ（サーバーで動くコア）の土台として先に行った

この期間の開発の流れ

「分けてから移す」の順。先に核と中身の境目を作り、そのあと核を Rust へ、最後に見た目を整えた。

① 分ける
コアと中身の分離
- 大型構築物の「チャンク化」を廃止
- コアが街を知らない形に（5 段階）
- 機器の状態・セーブの形を決める

② まとめる
MOD への統合
- サバイバル・街を MOD に
- MOD 選択画面で選ぶ形に統一
- 古いワールドは自動で読み替え

③ 移す（段階A）
Rust の物理コア
- 形合わせ・結合・地形・重力など
- Python（numba）版へ自動で戻れる
- Windows 用のビルド手順

④ 移す（段階B）
1サブステップまるごと
- 構築物1つの計算を 2 回の呼び出しに
- 計算中は Python を止めない
- 構築物ごとに本当に並列

⑤ 測る
重さの内訳の表示
- fps・物理・描画準備・描画と待ち
- 10 秒ごとにログへ
- 遊ぶ PC の数字で次を決める

⑥ 軽くする
描画準備
- 止まっている物は一度だけ作る
- 動く形の頂点は Rust でまとめて
- 14.0 → 7.3 ms／フレーム

⑦ 直す
見つけた不具合
- エディタで開けない（Windows）
- 遠景が穴や池に重なる
- 見本ロケットが止まる、ほか

⑧ 整える
くっきりした見た目
- ピクセルアートのテクスチャ
- 輪郭の線と角の陰
- 回転部分のねじれ解消

70 ファイル
変更したファイル（追加 7,599 行・削除 2,299 行）

2,757 行
Rust の物理コア（6 つのモジュール）

35 件
自動テスト（コア 14・Rust と Python の一致 20・街 1）

コアと中身を分ける — Rust 化とサーバー化の前提づくり

フェーズ1 の時点では、物理やワールドの中に街の処理が入り込んでいた。コアが街や画面を知らなくても動くように、5 段階で切り分けた。

区分	場所と中身
コア	main/ 直下：ワールド・構築物と物理・機器・惑星・プレイヤー・無線。画面も MOD も読み込まない
コアの決まりごと	hooks（MOD の口）・modcode（MOD の読み込み）・devstate（機器の状態の形）・savefmt（セーブの形）・control（スクリプトの窓口）
Rust の物理コア	native/alpedia_core。無ければ Python 版で動く
MOD	mods/：定義（JSON）とプログラム。本体同梱はサバイバルと街
画面	main/client/：描画と操作。MOD の画面は専用の口で足す

コアが画面や MOD を読み込んでいないことは、自動テストが毎回確かめる。

切り分けた 5 段階

- **1.** コアが街に頼らない：MOD の口・図形の計算・構築物を「枠」に乗せる仕組み
- **2.** 街の画面を MOD 側へ移す
- **3.** 機器の状態の形と、セーブの形（版番号つき）を決める
- **4.** スクリプトは窓口（Port）だけを通して構築物を読み書き
- **5.** チャンク化・古いエディタなど使わない部分を整理

マルチプレイへの効き目

- コアだけをサーバーで動かせる（画面なしで動くことをテストで確認済み）
- スクリプトが触れる範囲が窓口で決まっているので、サーバー側で実行しやすい

サバイバルと街を MOD に ― 「拡張」と「MOD」を1つに

これまでは「拡張（addons）」と「MOD」の2つの仕組みがあった。サバイバル（資源のルール）と街を MOD にまとめ、ワールドを作るときに MOD 選択で入れる形にした。



新しいワールドの画面。サバイバル・街・サンプル MOD を押して切り替える（サバイバル無し＝クリエイティブ）

MOD のプログラム

- mods/<名前>/code/ に置く。コアの口（読み込み・保存、物理の前後、当たり判定の箱、視線、E の操作、支払い、掘る、ジェットパックなど）を必要な分だけ書く
- 画面の口（キー・狙い・毎フレーム）も別に用意

今までのワールドもそのまま

- セーブ形式を format 2 にし、古い「クリエイティブ／街あり」の設定は自動で MOD に読み替え
- 既存のワールド（パッケージ版 world1）を読み込み、サバイバル＋街・所持金 30,254 がそのまま残ることを確認

遊ぶ人から見て

- 「サバイバル要素や街が欲しいときは MOD を選ぶ」の1つの考え方で済む
- 他の人が作った MOD も同じ場所・同じ選び方

Rust の物理コア（段階A） — 重い部品から置き換える

まず、numba で機械語化していた「重い部品」を Rust に置き換えた。ゲームの流れは Python のまま、部品だけが Rust で動く。

Rust にした部品	Rust の場所	元の Python
ボクセルの形合わせ・結合（複数コアで並列）	`kernels.rs`	`fastphys.py`
当たり判定の芯・描画用の頂点と法線・向き付きの箱	`kernels.rs`	`fastphys / geom`
完全剛体モードの関節つき剛体	`art.rs`	`fastphys.py`
惑星の地形（距離場・池・洞窟・掘削跡）と地面との接触	`planet.rs`	`planet / terrain_earth`
重力・構築物どうしの近い点の組	`lib.rs`	`world.py（scipy）`

安全に入れ替える仕組み

- `lib/` にビルドしたものがあれば自動で使う（画面左上に「物理:Rust」）
- 無ければ今までの Python（numba）版。設定 `physics.backend` で auto / rust / python を選べる
- Rust で計算している間は Python を止めない（ほかのスレッドが進める）

段階Aの結果

- チュートリアル：構築物の物理 **59.3 → 25.0 ms**／ティック（2.4 倍）
- ドローンの飛行記録は Python 版と**完全に同じ**。振り子の誤差 0.55°、ロボットの関節のずれ 1.3 mm で、フェーズ1 と同じ
- ただし構築物1つあたり十数回の呼び出しが残り、呼び出しの手間が目立った → 段階B へ

段階B：1サブステップをまるごと Rust で

構築物1つの1サブステップ（1/180 秒）の計算を、Rust の呼び出し 2 回（前半・後半）にまとめた。配列は Python が持ったまま、Rust がその場で書き換える。

これまで（段階A）



灰：Python（numpy） 青：Rust
行き来が多く、その間は他の構築物を止める

いま（段階B）



呼び出し 2 回だけ。計算中は Python を止めないので、小さな構築物どうしが複数コアで同時に進む

Rust に入れたもの

- ブロックごとの計算・剛体・完全剛体（接触をまとめて解く非負最小二乗も自作）
- 形が変わるまで変わらない値（結合・質量・回転関節）は「Body」として一度だけ作る

Python に残したもの

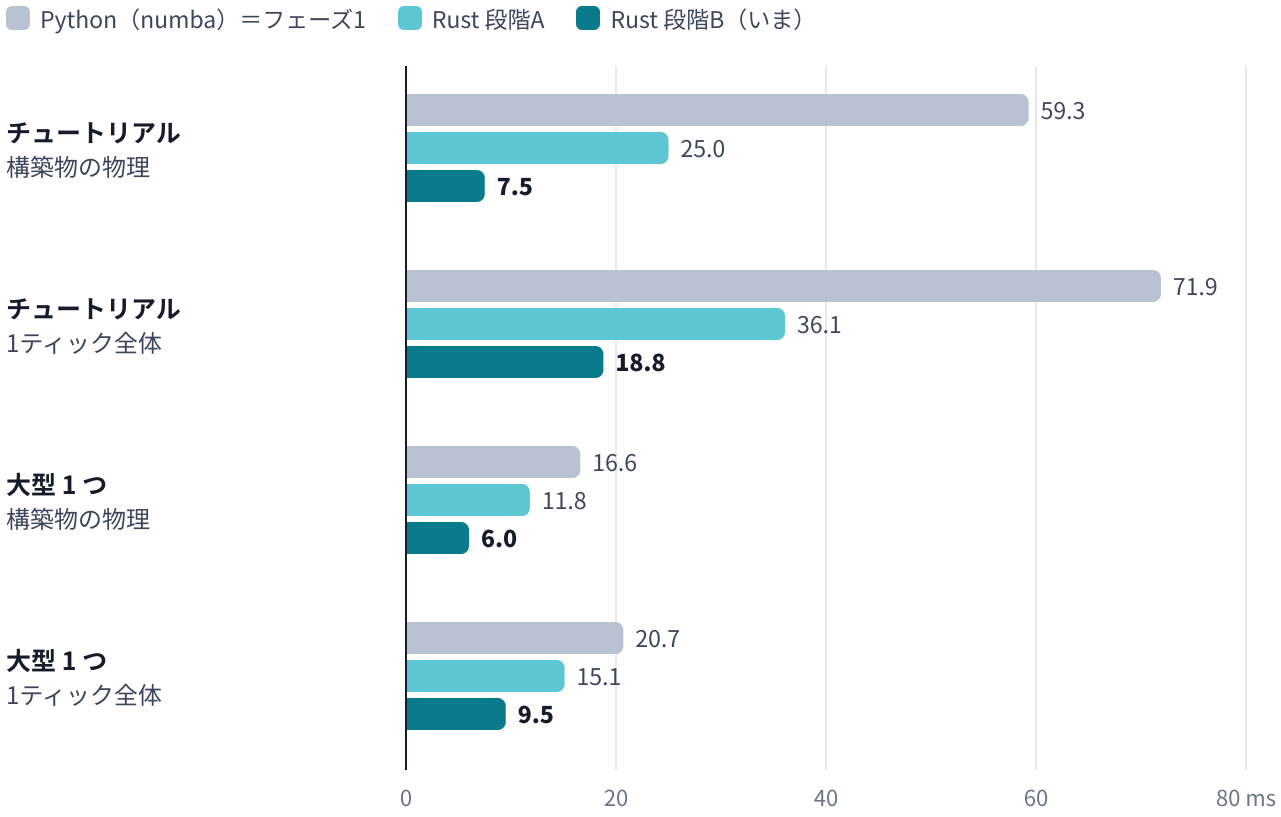
- 街など MOD の当たり判定の箱（MOD のプログラムを呼ぶため）
- ちぎれたあとの作り直し、関節つき剛体の流れ
- 環境変数 `ALPEDIA_NATIVE_STEP=0` で段階Aに戻して比べられる

効果

- チュートリアル：構築物の物理 25.0 → **7.5 ms**（段階Aの 3.3 倍）
- 遊ぶ PC でも Rust の新しい版が読み込まれ、動いていることをログで確認

速さの比較 — 同じ PC・同じ場面で 3 つの計算方法

1 ティック（1/60 秒、サブステップ 3 回）にかかった時間。ゲームの画面なしで 240 ティック動かした平均。



場面	物理（構築物）	1ティック全体
チュートリアル 構築物 36・655 ブロック	59.3 → 25.0 → **7.5 ms** **7.9 倍**	71.9 → 36.1 → **18.8 ms** **3.8 倍**
大型 1 つ 409 ブロック・回転中	16.6 → 11.8 → **6.0 ms** **2.8 倍**	20.7 → 15.1 → **9.5 ms** **2.2 倍**

読み方：小さな構築物がたくさんある場面ほど速くなる（呼び出しの手間が消え、複数コアで同時に進む）。大きな構築物1つは、もともと numba で並列だった形合わせが中心なので伸びが小さい。

1ティックの残り（チュートリアル 18.8 ms）：構築物の物理 7.5・構築物どうしの押し出し 5.5・その他（剛体にするかの判定・機器・スクリプト・プレイヤー）5.9。次はここを減らす。

計測：このレポート作成時に開発用の Linux 環境で計測（同じ PC・同じ条件で 3 方式を切り替え）。遊ぶ PC での数字は p.12。

同じ動きになっているか — 速さより先に確かめたこと

Rust にしても遊び心地が変わらないことを、部品・1サブステップ・場面全体の 3 つの段で比べた。

1e-7 以下
部品ごとの差（形合わせ・結合・芯・関節つき剛体・地形・重力・近い点の組）

1e-6 m 以下
1サブステップまるごとの差（8 場面 × 12 サブステップ）

2.6e-9 m
チュートリアルを 6 秒動かしたあとの、構築物 36 個の重心の最大の差

1サブステップの比較テスト（自動）	確かめること
地面に置いた板	地面との接触と摩擦
粘土の塊	曲がったまま戻らない変形（塑性）
アクチュエーターつき	回転関節の目標と分け方
引きはがした木のブロック	ちぎれる判定と作り直し
水に浮かべた中空の箱	浮力・水の抵抗・気密の空洞
剛体・剛体で水の上・完全剛体	剛体の接触・非負最小二乗

場面で比べる

- ドローンの飛行（高度・位置・燃料の記録）が同じ
- サバイバルの資源・お金の流れ、街の取引・損傷・保存と読み込みが同じ
- 既存のワールドを読み込んで保存しても中身が変わらない

いつでも戻せる

- Rust が無い・壊れていても Python 版で遊べる
- 比べるための切り替え：`ALPEDIA_BACKEND=python`／`ALPEDIA_NATIVE_STEP=0`

Windows でのビルドと配布 — ダブルクリック 1 回

Rust を入れたあとは native\build_windows.bat をダブルクリックするだけ。Visual Studio は要らない。

できるもの	使う Python
lib\alpedia_core.pyd	この PC の Python（3.10 以上ならどれでも）
…cp312-win_amd64.pyd（パッケージ版にも置く）	パッケージ同梱の Python 3.12 専用

遊ぶ PC で 2 種類あわせて約 3 分（1 分 22 秒＋1 分 28 秒）。記録は native\build_log.txt に残る。

最初の失敗と対策

- 1 回目は何も作られなかった。原因は、Python と結びつけるための道具（dlltool）が無いこと
- → GNU ツールチェーンにして、Python の DLL に直接つなぐ設定ファイルを渡す方式に変更
- パッケージ版の Python には共通の DLL（python3.dll）が無い → 3.12 専用版も作る
- 途中のファイルは OneDrive の外（%LOCALAPPDATA%）に置いて同期の衝突を防ぐ

配布への組み込み

- パッケージを作る手順が lib の Rust 版も自動でコピー
- Mac / Linux は同じソースからビルドできる（手順は README）。パッケージ化は次の課題

遊ぶ PC での重さ — 測ってみたら、次は描画だった

Rust にしても「思ったより速く感じない」という声から、画面左上とログに重さの内訳を出すようにした。遊ぶ PC（Intel 内蔵 GPU）のチュートリアルの記録：

```
描画準備の軽量化の前　重さ: fps 24  物理 7.7 ms/回×2.4  描画準備 9.0 ms  描画・待ち 12.7 ms  遅れ 4%（構築物 36, 物理:Rust）
描画準備の軽量化の後　重さ: fps 32  物理 6.7 ms/回×1.9  描画準備 3.9 ms  描画・待ち 13.3 ms（構築物 36, 物理:Rust）
```

わかったこと

- 物理 1 回は約 7 ms で、1/60 秒（16.7 ms）の半分以下に収まった
- 1 フレームが長いと物理を 2〜3 回まとめて回すので、描画が遅いと全体がさらに遅くなる
- 次の本命は **描画準備** と **GPU の描画（描画・待ち）**

描画準備を軽くした

- 構築物ごとに親ノードを 1 つにし、剛体・止まっている物は形・記号・看板を一度だけ作って、毎フレームは位置と向きだけ動かす
- ブロックごとに動く形は、頂点・法線・陰影を Rust でまとめて作る
- 開発環境 14.0 → **7.3 ms**、遊ぶ PC 約 9 → **約 4 ms**

残り

- チュートリアルの fps（10 秒ごとの値の中央値）は 24 → **30**。60 fps には届いていない
- 「描画・待ち」が 11〜20 ms：GPU の描画の回数・量を減らす必要がある
- 構築物の無い場所・少ない場所では 60 fps（描画・待ちの大半は 60 fps に合わせる待ち）

表示	意味	表示	意味
物理 ○ ms/回×○	物理 1 回（1/60 秒ぶん）の時間と、1 フレームで回した回数	**描画・待ち**	GPU の描画と、60 fps に合わせる待ちの時間
描画準備	描画の前に Python がする処理（構築物の形・記号・看板）	**遅れ**	処理が追いつかず、ゲーム内の時間が遅れた割合

くっきりした見た目 — テクスチャ・輪郭の線・角の陰

形（丸いブロック）はそのままに、貼り方と描き方を変えた。物理と当たり判定には手を入れていない。



左：前　右：いま（機器の見本と台座）。岩の石組み・機器の外板・ブロックの境目の線が見える



新しいテクスチャ（16×16）：金属・岩・木・ガラス・ゴム・粘土・水・機器の外板・輪郭の線

ぼやけの原因を直す

- 前は頂点ごとに投影の向きを決めていて、向きの変わり目の三角形でテクスチャが引き伸ばされていた
- いまは三角形ごとに投影し、頂点は向きごとに持ち分ける（位置と法線は共通）

読みやすさ

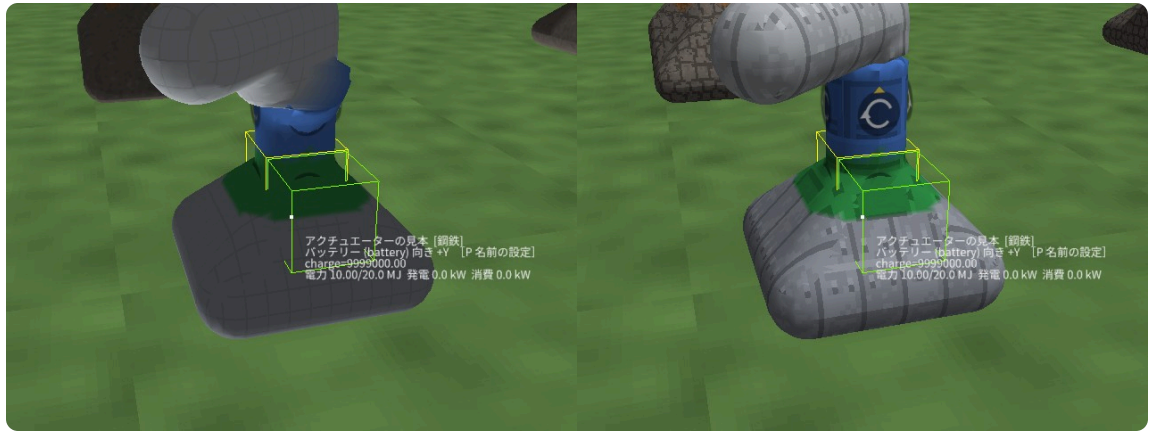
- ブロックの境目に輪郭の線（設定 `block_edges`）
- くぼんだ所に軽い陰（設定 `block_ao`）
- テクスチャの無い機器は「機器の外板」を使う

ついでに見つけた不具合

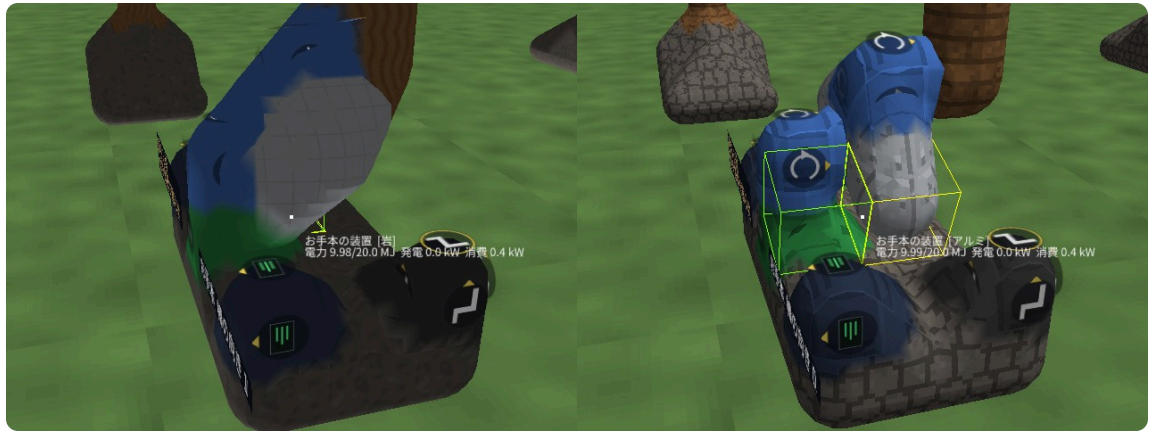
- 構築物の面の向きが内向きに計算されていて、日の当たる上の面のほうが暗かった → 修正

アクチュエーターを回してもねじれない

前は構築物全体を 1 つの形として描いていたので、回る部分とのつなぎ目が引き伸ばされ、ねじれて見えた。



アクチュエーターの見本を 45° 回したところ（左：前、右：いま）



お手本の装置（関節 2 つ）を動かしたところ（左：前、右：いま）

回る部分ごとに別の形

回転関節で分かれる部分ごとに形とメッシュを作る。境はブロックの境の面でまっすぐ切り、両側の切り口がぴったり合うようにした。

部分どうしを埋めない

別に動く部分どうしをつなぐ「埋めの形」は作らない。回したときに板のように飛び出さない。アクチュエーターと出力のつなぎ目だけは残す。

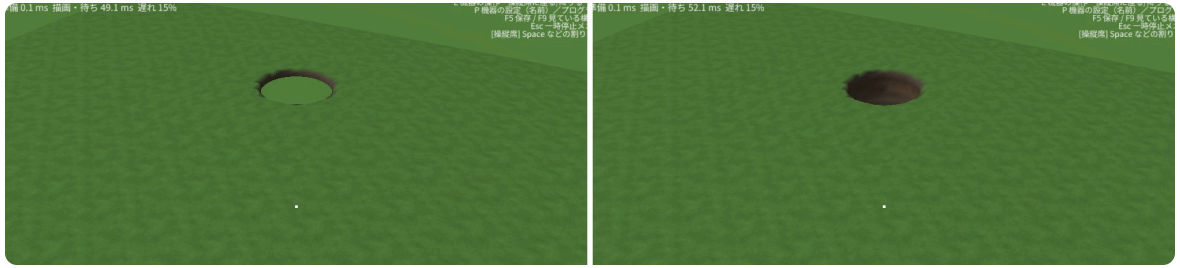
ほかへの効き目

アクチュエーターの記号が見えるようになった。回る部分は剛体として描くので、のちのマルチプレイで「部分ごとの姿勢」を送る考え方とも合う。

見つけて直した不具合

遊んでもらった中で見つかったもの、ログから見つけたもの。

症状	原因と直し方
「エディタで開く」で開かない	Windows では .py を「開く」と Python で実行されてしまう。VS Code、無ければメモ帳で開く（設定 `game.editor` で変更可）
掘った穴や池に緑の板が重なる	遠景用の粗い球が、詳しい地面の範囲の一部でくぼみの上に残っていた。詳しい地面の下では遠景を 60 m 沈める
見本ロケットのスクリプトが止まる	脱出軌道で遠点が無限大になり、ブロックの「四捨五入」が失敗。無限大はそのまま表示
上の面が暗い	面の向きが内向き。閉じた面の向きで外向きにそろえる
回すと形がねじれる	p.14 のとおり、回る部分ごとに別の形



約 38 m 先に掘った穴（左：前は緑の円盤でふさがる、右：いま）

この期間に判断したこと

進め方を変えた点と、その理由。

判断	理由	影響
大型構築物の「チャンク化」をやめる	関節つき剛体と Rust 化で、4×4×4 ごとの剛体に分ける必要が薄れた。コードが単純になる	300 ブロック以上もほかと同じ計算。速さは Rust で補う
拡張（addons）を MOD に統合	仕組みが 2 つあると分かりにくい。サバイバル・街も「選ぶ MOD」の 1 つにする	古いワールドは自動で読み替え。MOD 作者は 1 つの書き方で済む
Rust は段階的に（A → B）	部品だけ（A）→ 1 サブステップまるごと（B）。毎段階で Python 版と比べ、いつでも戻せる	段階 C（ワールドの 1 ティック）・D（Python なし）は効果を見て決める
Windows は GNU ツールチェーン	Visual Studio を入れなくてもビルドできる。パッケージ版の Python に合わせた版も作れる	ダブルクリックでビルド。ビルドは約 3 分
ブロックの形は丸いまま	箱型にすると、当たり判定（1 個なら球、並ぶと角が半径 0.5 m の丸）と見た目がずれ、転がり方が見た目と合わなくなる	くっきりさはテクスチャ・線・陰と、回る部分の分割で出す。ロードマップの「角の丸みを小さく」は見送り

フェーズ2 前半の判断基準の進み具合

フェーズ1 報告で決めた「この期間の終わりの判断基準」（〜2027 Q1）に対して。

Rust 核の物理が Python 版と同じ動きで 5 倍以上速い

✓ ほぼ達成

- 同じ動き：達成（p.10）
- チュートリアル（小さな構築物が多い）：**7.9 倍**
- 大型 1 つ（409 ブロック）：**2.8 倍** にとどまる
- 残り：大型の形合わせの並列の工夫、構築物どうしの押し出しの Rust 化

LAN で 4 人が同じワールドで 30 分遊んでも、ずれや切断がない

○ 未着手

- コアを画面なしで動かせる・スクリプトの窓口が決まっている、など土台はできた
- 次の 4 週間で 2 人の最小試作から始める
- 送る内容は「部分ごとの剛体の姿勢＋変形の差分」の予定

初めての人の 8 割が、説明なしでブロックの形を見分けられる

△ 一部

- 見た目の改善（テクスチャ・線・陰・ねじれ解消）は実装済み
- 人に見てもらうテストはまだ。5 人以上で行う
- 見分けにくい形（斜面と階段など）は結果を見て直す

見立て　柱1 と柱3 は前倒し、柱2 は遅れ。判断基準そのものは変えず、次の 4 週間は柱2 の試作と、柱3 の「人で確かめる」に重みを移す。

残っている課題 — どこが重いか・何が足りないか

数字で見た「次に効く所」。

チュートリアル 1 ティック（18.8 ms）の内訳（開発環境）



Rust（段階B）　構築物どうし（Python）　剛体化の判定・機器・スクリプト

押し出しとその他（11.4 ms）は Python のまま。ここが次の Rust 化の候補（段階C の入口）

遊ぶ PC の 1 フレーム（チュートリアル、32 fps のとき）



描画が長いと物理をまとめて回す回数が増える。描画と物理の両方を減らすと効き目が重なる

柱1

- 構築物どうしの押し出し・剛体化の判定が Python のまま
- 大型 1 つの速さ（2.8 倍）
- WASM・Mac/Linux のパッケージが未着手
- 同じ入力で同じ結果の保証（再現テスト）

柱2

- マルチプレイの試作が未着手
- セーブの「ワールド／プレイヤー」分け

柱3

- GPU 側の重さ（描画・待ち 11〜20 ms）
- ヒント表示・デザインの決まり・シナリオ
- テクスチャは第一版。人の目で調整

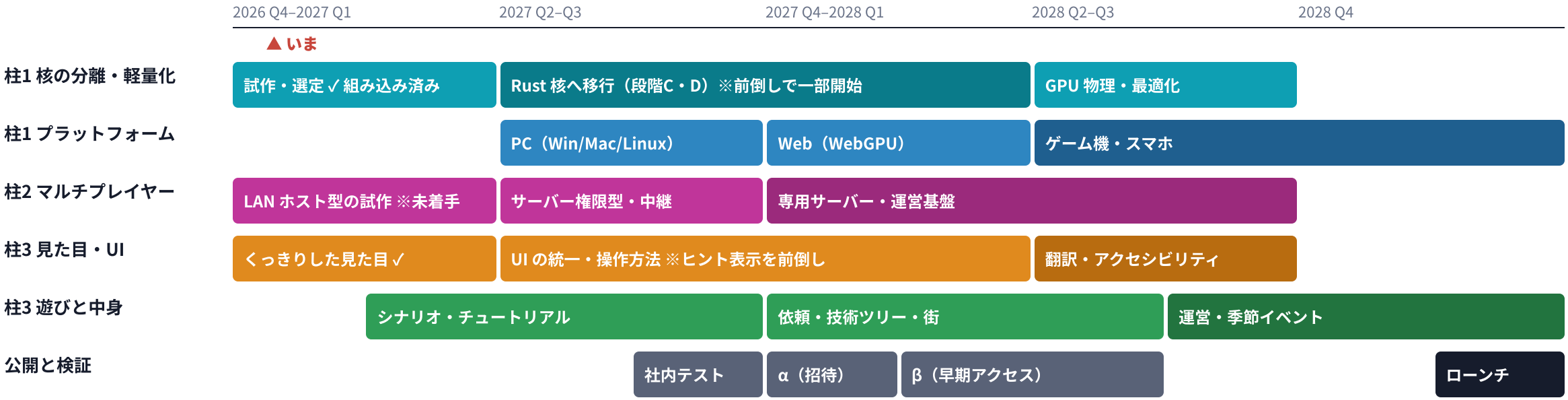
次の 4 週間でやること

それぞれに「終わったと言える目安」を付けた。

柱	やること	終わったと言える目安
柱1	構築物どうしの押し出しと剛体化の判定を Rust へ（段階C の入口）	チュートリアル 1 ティック 18.8 → **10 ms 以下**、Python 版と同じ動き
柱1	同じ入力で同じ結果の再現テスト（計算の順を固定）	Rust 版で 2 回動かして **1,000 ティック**一致
柱1	物理の芯を WASM でビルドしてブラウザで動かす試作	1サブステップの比較テストがブラウザでも通る
柱3	GPU 側の重さを減らす（描画の回数を数える、遠い構築物をまとめる・簡略化）	遊ぶ PC でチュートリアル **45 fps 以上**（最終目標 60 fps）
柱2	LAN ホスト型マルチプレイの最小試作（2 人で同じ構築物を動かす）	2 人で 10 分、ずれ・切断なし。通信量と遅延を「重さ」表示に出す
柱2	セーブを「ワールド」と「プレイヤー」に分ける（format 3、古い形式は自動で読み替え）	今のワールドがそのまま読める
柱3	画面下のヒント表示（狙った物の説明と今押せるキー）、色・余白・ボタンの決まり	主要 3 画面（ゲーム・インベントリ・コンピューター）に適用
柱3	形を見分けるテスト（初めての人 5 人以上）と、テクスチャの調整	8 割が見分けられるか判定。だめな形を直す
柱3	シナリオ第 1 章（墜落）の台本	章の目的・手順・案内役のセリフがそろう

ロードマップの更新 — 全体の日程は変えず、中身の順番を入れ替える

帯の位置はフェーズ1 報告の計画のまま。▲：いま（2026 年 10 月中旬）。帯の中の ✓・※ がこの期間の変化。



前倒し：Rust 核への移行（段階 B まで完了）、くっきりした見た目、WASM の試作、ヒント表示

遅れ：LAN ホスト型の試作。次の 4 週間で着手し、2027 Q1 末までに 4 人・30 分の基準を目指す

変えない：α（2027 Q3 末）・ローンチ（2028 Q4）の時期と、各フェーズの判断基準

リスクと対策（更新）、そしてまとめ

リスク	対策（この期間の変化）
Python と Rust の二重管理	一致テスト 20 件を常に回す。切り替えの環境変数で同じ場面を比べられる。移せていない所は Python のまま遊べる
描画（GPU）が新しい詰まり所	重さの内訳の表示で特定済み。次の 4 週間で描画の回数・量を減らす
PC ごとのビルドの違い	Windows は Visual Studio 不要の手順と記録。パッケージ版用の Python 3.12 専用版も作る。Mac/Linux のパッケージ化は次の課題
見た目と当たり判定のずれ	形は丸いままにする判断（p.16）。くっきりさは描き方で出す
マルチプレイの遅れ	コアの分離で土台はできた。最小試作に重みを移し、判断基準（4 人・30 分）は維持
やわらかい物理の同期が重い	回る部分を剛体として扱う作りが描画にも入った。送るのは部分の姿勢と差分の予定のまま

この期間の成果物

- Rust の物理コア（`native/alpedia_core`）と Windows 用ビルド手順
- コア／MOD／画面の分離、サバイバル・街の MOD
- ピクセルアートのテクスチャ 9 枚、重さの表示
- 自動テスト 35 件、README の更新

まとめ　フェーズ2 前半の折り返しで、**核の分離と Rust 化**は計画より先へ進み、ゲーム全体で 3.8 倍、構築物の物理で 7.9 倍速くなった。見た目も、形の手触りを残したまま**くっきり**させた。次の 4 週間は、残る重さ（構築物どうし・GPU）を減らしつつ、遅れている**マルチプレイの最小試作**と、見た目を**人で確かめる**ことに力を移す。